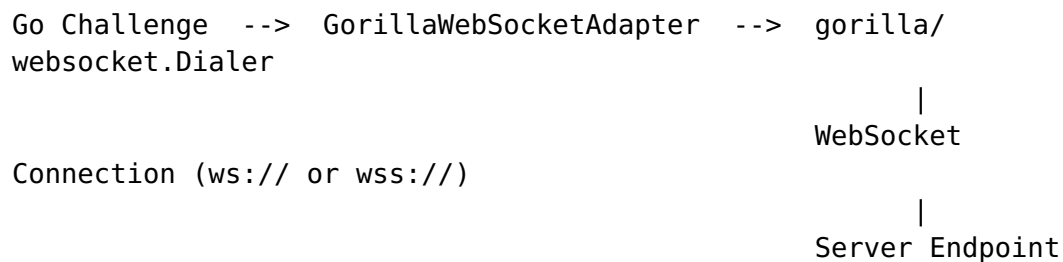


websocket-adapter

WebSocket Adapter

The WebSocket adapter defines a `WebSocketFlowAdapter` interface and provides an implementation (`GorillaWebSocketAdapter`) built on the `gorilla/websocket` library. It supports bidirectional messaging, timeout-aware receive operations, and flow-based testing patterns.

Architecture



Unlike the gRPC adapter (which shells out to a CLI tool), the WebSocket adapter maintains a persistent connection within the Go process. This enables true bidirectional communication with thread-safe read and write operations.

Thread Safety

The adapter uses two separate mutexes:

- `writeMu` protects `WriteMessage` calls (used by `Send` and `Close`)
- `readMu` protects `ReadMessage` calls (used by `Receive` and `ReceiveAll`)

This allows concurrent send and receive operations without deadlock.

WebSocketFlowAdapter Interface

Defined in adapter_websocket_flow.go:

```
type WebSocketFlowAdapter interface {  
    Connect(ctx context.Context, url string, headers  
        map[string]string) error  
    Send(ctx context.Context, message []byte) error  
    Receive(ctx context.Context, timeout time.Duration) ([]byte,  
        error)  
    ReceiveAll(ctx context.Context, timeout time.Duration) ([]  
        []byte, error)  
    SendAndReceive(ctx context.Context, message []byte, timeout  
        time.Duration) ([]byte, error)  
    Close(ctx context.Context) error  
    Available(ctx context.Context) bool  
}
```

Method Summary

Method	Description
Connect	Establishes a WebSocket connection with optional headers
Send	Sends a text message (thread-safe)
Receive	Reads the next message with a timeout
ReceiveAll	Reads messages until timeout, returns all collected
SendAndReceive	Sends a message and waits for a single response
Close	Sends close frame and terminates the connection
Available	Returns true if currently connected

Constructor

```
adapter := userflow.NewGorillaWebSocketAdapter()
```

No arguments. Call Connect before using other methods.

API Reference

Connect

```
err := adapter.Connect(ctx, "ws://localhost:8080/ws",  
    map[string]string{  
        "Authorization": "Bearer <token>",
```

```
"X-Request-ID": "test-123",
})
```

Establishes a WebSocket connection using gorilla/websocket.Dialer with a 10-second handshake timeout. Optional headers are added to the HTTP upgrade request. A pong handler is installed to keep the connection alive with a 60-second read deadline.

Returns an error if already connected (call Close first to reconnect).

Send

```
err := adapter.Send(ctx,
    []byte(`{"action":"subscribe","channel":"orders"}`))
```

Sends a text message over the WebSocket connection. Thread-safe via the write mutex. Returns an error if not connected.

Receive

```
msg, err := adapter.Receive(ctx, 5*time.Second)
```

Reads the next message from the WebSocket connection. Sets a read deadline based on the timeout. Returns an error if the deadline expires before a message arrives. Thread-safe via the read mutex.

ReceiveAll

```
messages, err := adapter.ReceiveAll(ctx, 10*time.Second)
```

Reads messages continuously until the timeout expires. All messages received within the window are returned. A timeout is not treated as an error -- it simply terminates collection. Useful for consuming server-push events or broadcast messages.

SendAndReceive

```
response, err := adapter.SendAndReceive(
    ctx,
    []byte(`{"action":"ping"}`),
    5*time.Second,
)
```

Convenience method that sends a message and waits for a single response. Equivalent to calling Send followed by Receive.

Close

```
err := adapter.Close(ctx)
```

Sends a WebSocket close frame with `CloseNormalClosure` status, then closes the underlying connection. Thread-safe via the write mutex. Safe to call multiple times (no-op if already closed).

Available

```
ok := adapter.Available(ctx)
```

Returns true if the adapter currently holds an active connection. Does not attempt to reconnect.

SSE vs WebSocket

The adapter is designed for WebSocket (bidirectional) connections. For Server-Sent Events (SSE), use the `HTTPAPIAdapter` with a streaming response parser. Key differences:

Feature	WebSocket	SSE
Direction	Bidirectional	Server-to-client only
Protocol	<code>ws://</code> / <code>wss://</code>	<code>http://</code> / <code>https://</code> (text/event-stream)
Client sends	<code>Send()</code> method	Separate HTTP requests
Reconnection	Manual (call <code>Connect</code> again)	Built into <code>EventSource</code> API
Binary data	Supported	Text only
Adapter	<code>GorillaWebSocketAdapter</code>	<code>HTTPAPIAdapter</code>

WebSocketFlowChallenge

The `WebSocketFlowChallenge` template executes a multi-step WebSocket flow:

```
challenge := userflow.NewWebSocketFlowChallenge(  
    "CH-WS-001",  
    "WebSocket Chat Flow",  
    "Verify real-time chat over WebSocket",  
    nil,  
    adapter,  
    userflow.WebSocketFlow{  
        URL: "ws://localhost:8080/ws/chat",  
    },  
)
```

```

Headers: map[string]string{
    "Authorization": "Bearer <token>",
},
Steps: []userflow.WebSocketStep{
    {
        Name:    "subscribe",
        Action:  "send",
        Message: `{"action":"join","room":"general"}`,
    },
    {
        Name:    "wait-for-ack",
        Action:  "receive",
        Timeout: 5 * time.Second,
        Assertions: []userflow.StepAssertion{
            {Type: "response_contains", Value: "joined",
Target: "ack"},
        },
    },
    {
        Name:    "send-message",
        Action:  "send_receive",
        Message: `{"action":"message","text":"hello"}`,
        Timeout: 5 * time.Second,
        Assertions: []userflow.StepAssertion{
            {Type: "not_empty", Target: "echo"},
        },
        ExtractTo: map[string]string{
            "id": "message_id",
        },
    },
    {
        Name:    "collect-events",
        Action:  "receive_all",
        Timeout: 3 * time.Second,
        Assertions: []userflow.StepAssertion{
            {Type: "message_count", Value: 1, Target:
"events"},
        },
    },
    {
        Name:    "pause",
        Action:  "wait",
        Timeout: 1 * time.Second,
    },
},
),

```

Step Actions

Action	Description	Fields Used
send	Send a message	Message
receive	Receive one message	Timeout
send_receive	Send and wait for response	Message, Timeout
receive_all	Collect all messages until timeout	Timeout
wait	Sleep for the timeout duration	Timeout

Flow Features

- **Variable substitution:** Use `{{var_name}}` in message payloads
- **Value extraction:** `ExtractTo` maps JSON response fields to variables
- **Step assertions:** `response_contains`, `not_empty`, `message_count`
- **Progress reporting:** Each step reports progress to the liveness monitor
- **Per-step metrics:** Duration tracked for each step
- **Auto-cleanup:** Connection is closed via `defer` after flow execution

Default Timeout

Steps without a `Timeout` default to 5 seconds.

Error Handling

Timeout detection uses string matching on the error message, checking for "timeout", "deadline exceeded", and "i/o timeout". This avoids importing the `net` package just for the `net.Error` interface. The `ReceiveAll` method treats timeouts as normal termination (not errors).

Source Files

- Interface + implementation: `pkg/userflow/adapter_websocket_flow.go`
- Challenge template: `pkg/userflow/challenge_websocket_flow.go`